

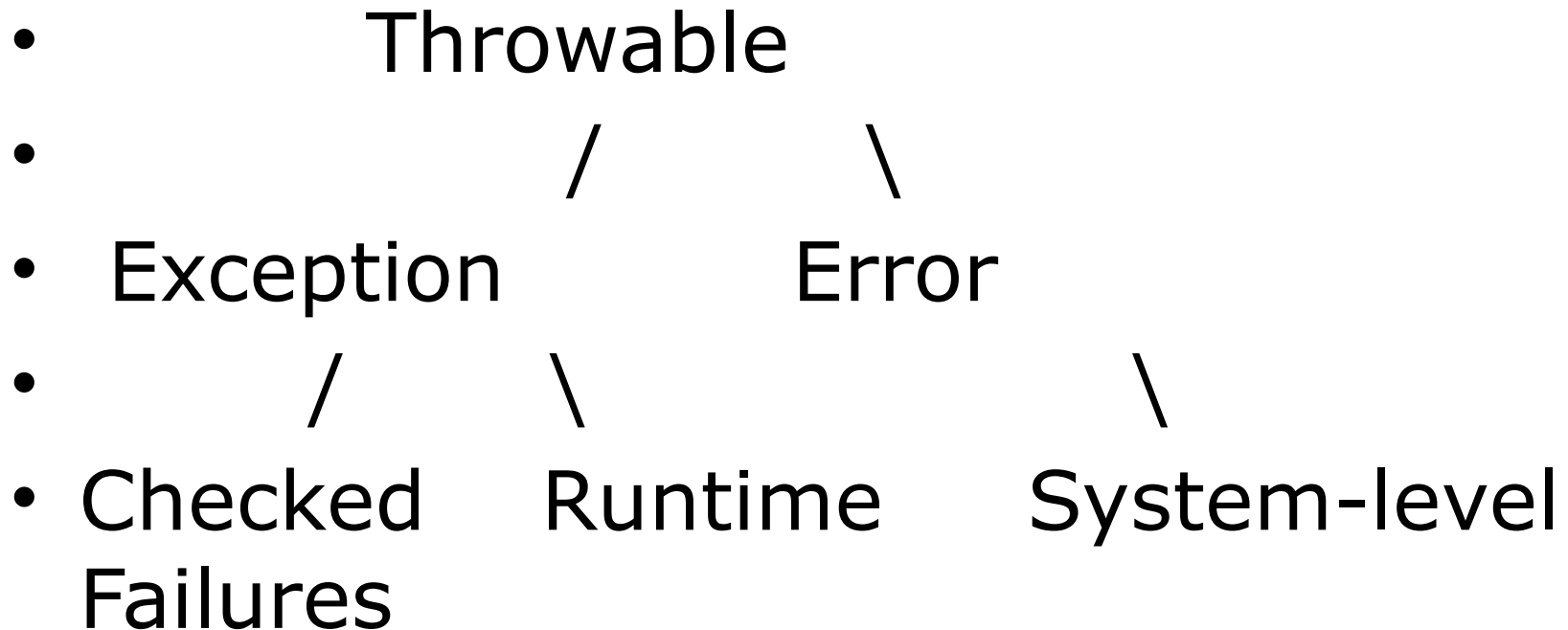
Exceptions in Java

Exception Hierarchy, Throwing
& Catching, Built-in & Custom
Exceptions

Introduction to Exceptions

- An exception is an event that disrupts normal program execution.
- Exceptions are handled using try, catch, and finally blocks.
- Java provides built-in exceptions, and users can define custom exceptions.

Exception Hierarchy



1. Throwable (Root Class)

The superclass for all errors and exceptions in Java.

It has two direct subclasses: **Exception** and **Error**.

2. Exception (Handled via try-catch)

Represents conditions that a program should catch and handle.

Further divided into **Checked** and **Unchecked (Runtime) Exceptions**.

- **a) Checked Exceptions**

- These must be handled using try-catch or declared using throws.
- Examples:
 - IOException (e.g., file not found)
 - SQLException (e.g., database connection issue)
 - InterruptedException (e.g., thread interruption)

- **b) Unchecked (Runtime) Exceptions**

- These occur due to programming errors and do not need explicit handling.
- Subclass of RuntimeException.
- Examples:
 - NullPointerException (Accessing an object reference that is null)
 - ArithmeticException (Division by zero)
 - ArrayIndexOutOfBoundsException (Invalid array index)

- **3. Error (Severe Issues)**
- Represents system-level errors that the program cannot recover from.
- Should not be caught using try-catch.
- Examples:
 - OutOfMemoryError (Heap space exhausted)
 - StackOverflowError (Recursive method calls overflow stack)
 - VirtualMachineError (JVM internal errors)

Exception Handling

- Exception Handling includes
 - Find the exception(Hit the exception)
 - Inform that an error has occurred(Throw the exception)
 - Receive the error information (Catch the exception)
 - Take corrective actions(Handle the exception)

Sample Code - Try-Catch

- `try {`
- `int result = 10 / 0; // Causes ArithmeticException`
- `} catch (ArithmeticException e) {`
- `System.out.println("Exception caught: " + e);`
- `}`

Built-in Exceptions

- • NullPointerException - Accessing an object reference that is null.
- • ArithmeticException - Arithmetic operation error (e.g., division by zero).
- • ArrayIndexOutOfBoundsException - Accessing an invalid array index.

Creating Custom Exceptions

- Custom exceptions extend `Exception` or `RuntimeException`.
- Used to define application-specific error handling.

Sample Code - Custom Exception

- class MyException extends Exception {
- public MyException(String message) {
- super(message);
- }
- }
-
- public class Test {
- public static void main(String[] args) {
- try {
- throw new MyException("Custom Exception Occurred");
- } catch (MyException e) {
- System.out.println(e.getMessage());
- }
- }
- }

Conclusion

- • Exceptions help in robust error handling.
- • Use try-catch-finally and throw/throws effectively.
- • Built-in exceptions cover common cases, but custom exceptions allow specific handling.